SOFTWARE ENGINEERING
& PROJECT MANAGEMENT

DAY 7

# The Agile Model

A student-facing chapter on iterative development, collaboration, responding to change, and practical delivery

**Module I: Introduction to Software Engineering**
Topic: Software Process Models — Agile Model
Duration: 1 hour

**Running example:** College Event Management System (CEMS)

Understand how teams organize work into short cycles, collaborate with stakeholders, deliver useful software frequently, and adapt plans as they learn.

# 1. Learning Objectives

By the end of this chapter, you should be able to:

- Define Agile software development and explain its central ideas.
- Describe iterative and incremental delivery in an Agile context.
- Explain how collaboration, feedback, and responding to change influence planning.
- Apply Agile ideas to a realistic software project.
- Compare Agile with Waterfall, Incremental, Spiral, and Prototype approaches.
- Identify common misconceptions and challenges in Agile adoption.

**Key idea**
Agile is an approach to software development that emphasizes delivering useful software, collaborating with customers and teammates, learning through feedback, and adapting plans when requirements change.

# 2. Why Did Agile Become Important?

In many software projects, requirements evolve after users see a working system, market conditions change, or new technical information becomes available. A plan written at the beginning may no longer describe the most valuable work several months later.

Consider the College Event Management System. At first, the college may ask only for event listings and registrations. After trying the initial version, coordinators may request waitlists, QR-code attendance, or event reminders. Agile practices help a team review priorities regularly instead of treating the initial plan as unchangeable.

Agile does not mean “no planning” or “no documentation.” It means planning and documentation should support delivery and learning, with detail appropriate to the project's needs and risks.

# 3. What Is Agile Software Development?

Agile software development is a family of approaches based on iterative work, frequent delivery, collaboration, continuous feedback, and responding to change. Rather than trying to predict every detail perfectly at the start, a team delivers work in manageable portions and uses what it learns to guide subsequent work.

Agile is not one single process model with one mandatory set of meetings. Scrum and Extreme Programming (XP) are examples of approaches associated with Agile values and principles. In this course, Scrum will be studied in more detail in Module V.

**Important distinction**
Agile is a broad approach. Scrum is a specific framework that helps teams organize and manage work using defined roles, events, and artefacts.

# 4. Core Ideas of Agile

The Agile Manifesto expresses four value preferences. They do not mean the items on the right have no value; rather, the items on the left are valued more highly when making trade-offs.

| Agile values | Practical meaning |
|---|---|
| Individuals and interactions over processes and tools | Communication and teamwork matter; tools should support people rather than replace collaboration. |
| Working software over comprehensive documentation | A usable product is important, while necessary documentation still supports quality, safety, handover, and maintenance. |

| Agile values | Practical meaning |
|---|---|
| Customer collaboration over contract negotiation | Regular conversations help the team understand actual needs and make informed trade-offs. |
| Responding to change over following a plan | Plans are useful, but the team can revise priorities when new evidence or needs appear. |

## 5. Iterative and Incremental Development

These two terms are related but not identical:

- **Iterative:** the team repeats work to learn, improve, or refine a solution. For example, it improves the event-registration flow after observing students use it.
- **Incremental:** the team adds functionality in portions. For example, it releases event listings first, then registration, then notifications.
- **Agile work often combines both:** each short cycle can deliver a new capability and improve what already exists.

**Example**
An early registration page is delivered in one cycle (incremental delivery). After students test it, the team improves labels and error messages in the next cycle (iteration).

## 6. A Typical Agile Work Cycle

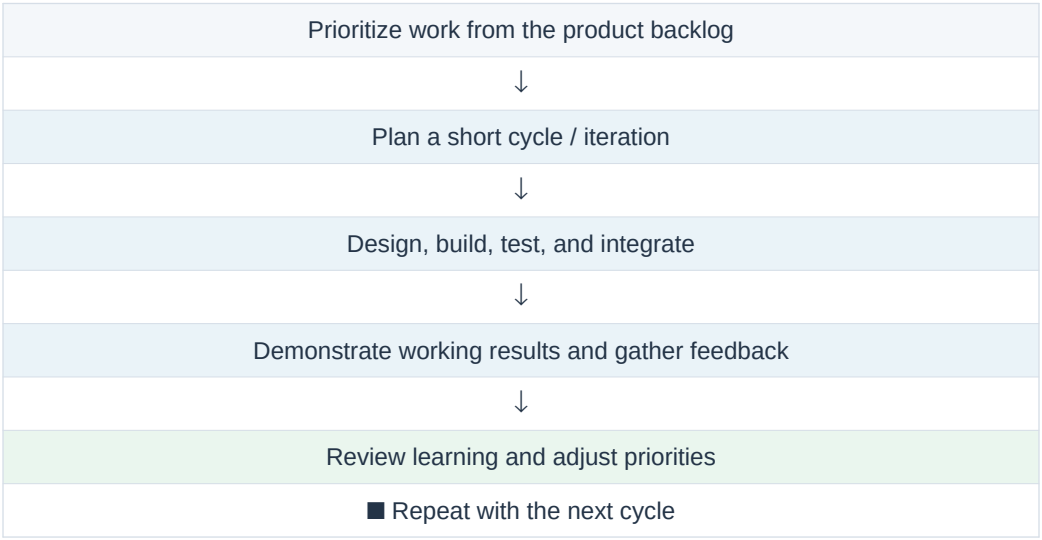


Figure 1. A simplified Agile cycle. Specific practices and cycle lengths vary by approach and team.

## 7. Worked Example: Agile Delivery for CEMS

The college wants an initial release before the annual festival, but some requirements are still uncertain. The team prioritizes features that provide immediate value and then improves the system through feedback.

| Cycle | Goal and sample work | Feedback / learning |
|---|---|---|
| 1 | Display events with dates, venues, eligibility, and organizer information. | Students ask for clearer filters and event capacity. |
| 2 | Add registration, cancellation, and confirmation. | Users want clearer confirmation and full-event behavior. |

| Cycle | Goal and sample work | Feedback / learning |
|---|---|---|
| 3 | Improve filters, add waitlist behavior, and refine messages. | Coordinators request a participant export. |
| 4 | Add attendance support after validating security and scanning needs. | Pilot results identify scanning or connectivity issues. |

This plan is illustrative, not a fixed formula. A team should not promise a feature in a cycle unless it understands the work, dependencies, and quality checks required.

## 8. Benefits of Agile

- **Frequent feedback:** users can evaluate working results and clarify priorities.
- **Earlier value:** useful features may be available before the full scope is complete.
- **Adaptability:** priorities can change as the team learns.
- **Transparency:** regular reviews help stakeholders see progress and obstacles.
- **Continuous improvement:** teams can inspect how they work and adjust their practices.

## 9. Challenges and Misconceptions

- **“Agile means no planning.”** Teams plan frequently and keep the plan current.
- **“Agile means no documentation.”** Teams create documentation that supports delivery, compliance, and maintenance.
- **“Agile means every request is accepted immediately.”** New requests must be prioritized against capacity, risk, and existing commitments.
- **“Agile guarantees faster delivery.”** Results depend on team capability, scope, technical quality, stakeholder availability, and constraints.
- **“Agile is only for small projects.”** Larger organizations can use Agile approaches, but coordination, architecture, and governance require attention.
- **Technical debt can accumulate.** Frequent delivery without sound engineering practices can make future changes harder.

**Agile still needs discipline**
Clear requirements for each piece of work, testing, integration, security, version control, and appropriate documentation remain important.

# 10. Agile Compared with Other Models

| Model | Organizing idea | Useful when... |
|---|---|---|
| Waterfall | Sequential phases with defined outputs. | Requirements are stable and formal phase gates are valuable. |
| Incremental | Product functionality is delivered in portions. | The system can provide value through staged feature releases. |
| Spiral | Repeated cycles explicitly guided by risk analysis. | Important technical or project risks need systematic attention. |
| Prototype | Early representations help clarify requirements or design. | Users or developers need to explore uncertain needs or feasibility. |
| Agile | Collaborative, iterative delivery with frequent feedback and adaptation. | Priorities and requirements may evolve and feedback is available. |

# 11. Classroom Activity: Prioritize Features

Scenario: The college festival is in three weeks. The team has limited time and is considering these features: event listings, registration, QR attendance, certificates, and advanced analytics.

In groups, rank the features by priority. Explain which feature should be delivered first, which can wait, what risks must be checked, and what feedback you would collect after the first release.

There is no single correct ordering without more context. Strong answers justify choices using user value, dependencies, deadline, risk, and effort.

# 12. Knowledge Check

1. What is Agile software development?
2. Name the four Agile Manifesto value preferences.
3. Explain the difference between iterative and incremental development.
4. Does Agile mean no planning or documentation? Explain.
5. Give two benefits and two challenges of Agile.
6. How can feedback change the CEMS product backlog?
7. What is the difference between Agile and Scrum?
8. Why are testing and technical quality still important in Agile?

## Answer key

1. An approach emphasizing collaboration, iterative delivery, feedback, and adapting to change. 2. Individuals/interactions over processes/tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan. 3. Iterative means refining through repeated cycles; incremental means adding functionality in portions. 4. No; Agile teams plan and document according to need. 5. Benefits: feedback and adaptability; challenges: coordination, changing priorities, technical debt. 6. New evidence may change the priority or detail of future work. 7. Agile is a broad approach; Scrum is a specific framework. 8. Without quality practices, frequent changes can introduce defects and make the product hard to maintain.

## 13. Key Terms

| Term | Meaning |
|---|---|
| Agile | A family of approaches emphasizing collaboration, feedback, iterative delivery, and adaptation. |
| Iteration | A repeated work cycle used to deliver or improve a solution. |
| Increment | A portion of product functionality added to the existing product. |
| Product backlog | A prioritized list of product work and desired changes. |
| Stakeholder | A person or group affected by or interested in the product. |
| Feedback loop | A process of reviewing results and using observations to guide future work. |
| Technical debt | Future cost created by shortcuts or design decisions that make later changes harder. |
| Scrum | An Agile framework with defined accountabilities, events, and artefacts. |

## 14. Homework / Preparation for Day 8

For a student portal, prepare a short Agile plan:

- Write a one-sentence product goal.
- List six possible features and prioritize them.
- Choose the first two features to deliver and justify the order.
- Write three questions to ask users after the first release.
- Identify one project risk and how the team will investigate it.

**Next chapter**
Day 8 will consolidate Module I by reviewing software engineering principles and software characteristics explicitly, then connecting them to the process models studied so far.

Syllabus alignment: Module I — Introduction to Software Engineering; Software Process Models: Agile. Scrum is introduced only at a high level here and will be covered in Module V.